DEPAR TMENT OF COMPU TER SCI ENCE AND ENGI NEER I NG

Subject Name: SOFTWAR E ENGINEER I NG **Subject code: CS T 55**

U NI T I I I

Software Design and Functi on Oriented Software Design: Outcome of a Design Process – Characteristi scs of a Good Software Design – Coupling and Cohesion – Approaches to Software Design – Object Oriented Vs Function Oriented Software Design approaches – Structured Analysis – Data Flow Diagrams – Applying DFD to Real time systems – Structured and Detailed Design.

.

2 MAR KS:

1 **What are items designed and documented duri ng design process**

- Different modules required
- Control relationships among modules
- Interfaces among different modules
- Data structures of the individual modules
- Algorithms required to implement the individual modules

2 **Classify design activi ties**

- Preliminary (high-level) design
- Detailed design

3 **Wri te about preli mi nary design**

- Through high-level design, a problem is decomposed into a set of modules, the control relationships among various modules identified, and also the interfaces among various modules are identified.
- The program structure or the software architecture is the outcome of high-level design

4 **Wri te about detailed design**

- During detailed design, each module is examined carefully to design its data structures and the algorithms
- The outcome of the detai led desi gn stage i s documented i n the form of a module

specification(MSPEC) document

5. Classify Design methodologies

- Procedural-oriented approach
- Object-oriented approach

6. Differentiate Analysis from design activities

- The goal of any analysis technique is to elaborate the customer requirements through careful thinking and the same time consciously avoiding making any decisions regarding the exact way the system is to be implemented.
- The design model reflects several decisions taken regarding the exact way system is to be implemented.
- In function-oriented approach, the **analysis model documented using data flow diagrams(DFD)**, whereas the **design documented using structure chart**.
- In object-oriented approach, both the design model and the analysis model documented using UML(Unified Modelling Language).

7. List characteristics of a good software design.

- Correctness
- Understandability
- Efficiency
- Maintainability

8. Write about design solution

- A design solution have the following features
  - It should assign consistent and meaningful names to various design components
  - It should make use of the principles of decomposition and abstraction in good measures to simplify the design
- A design solution is understandable, if it is modular and the modules are arranged in layers
- A design solution is considered to be highly modular, if the different modules in the solution have high cohesion and their inter-module couplings are low

9. Give difference between cohesion and coupling

- Cohesion is a measure of the functional strength of a module, whereas the coupling between two modules is a measure of the degree of interaction (or interdependence) between the two modules.

10. When do we say two modules are said to be highly coupled?

- If the function calls between two modules involve passing large chunks of shared data, the modules are tightly coupled.
- If the interactions occur through some shared data, then also we say that they are highly coupled.

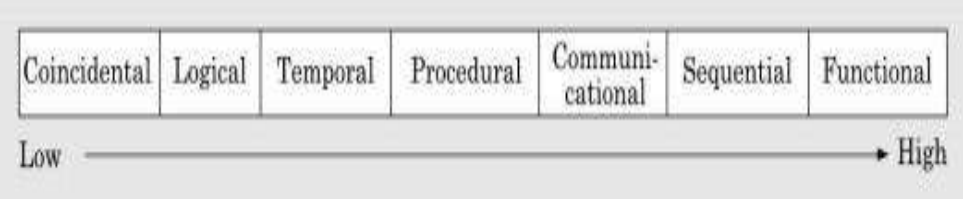
11. What is functional independence?

- A module that is highly cohesive and also has low coupling with other \modules is said to be functionally independent of the other modules.
- Advantages of functional independence
  - Error isolation
  - Scope of reuse
  - Understandability

12. Define cohesiveness of a module?

- Cohesiveness of a module is the degree to which the different functions of the module cooperate to work towards a single objective

13. Draw classification of cohesion? (or) what are different types of cohesion? (Nov 2017)



14. What is functional cohesion?

A module is said to posses functional cohesion, if different functions of the module cooperate to complete a single task.

15. What is Logical cohesion?

A module is said to be logically cohesive, if all elements of the module perform similar operations such as error handling data input, data output, etc.

16. What is Procedural cohesion?

A module is said to possess procedural cohesion, if the set of functions of the module are executed one after that other, though these functions may work towards entirely different purposes and operate on very different data.

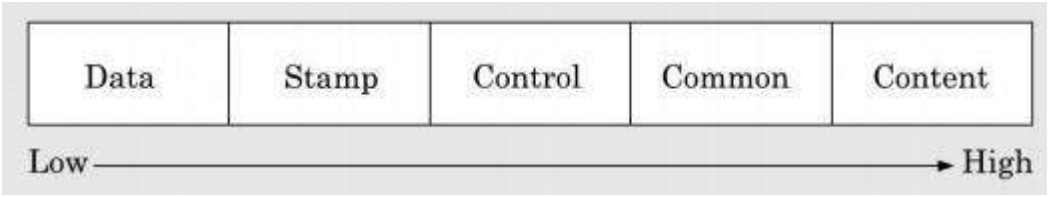
17. What is communicational cohesion?

A module is said to have communicational cohesion, if all functions of the module refer to or update the same data structure.

18. What is sequential cohesion?

A module is said to possess sequential cohesion, if the different functions of the module execute in a sequence, and the output from one function is input to the next in the sequence.

19. Give classification of coupling



20. What is data coupling?

Two modules are data coupled, if they communicate using an elementary data item that is passed as a parameter between the two, e.g an integer, a float, a character, etc.

21. What is Stamp coupling?

Two modules are stamp coupled, if they communicate using a composite data item such as a record in PASCAL or structure in C.

22. What is control coupling?
Control coupling exists between two modules, if data from one module is used to direct the order of instruction execution in another.

23. What is common coupling?
Two modules are common coupled, if they share some global data items.

24. What is content coupling?
Content coupling exists between two modules, if they share code. That is, a jump from one module into the code of another module can occur.

25. Define coupling
- The coupling between two modules indicates the degree of interdependence between them.
- The degree of coupling between two modules depends on their interface complexity

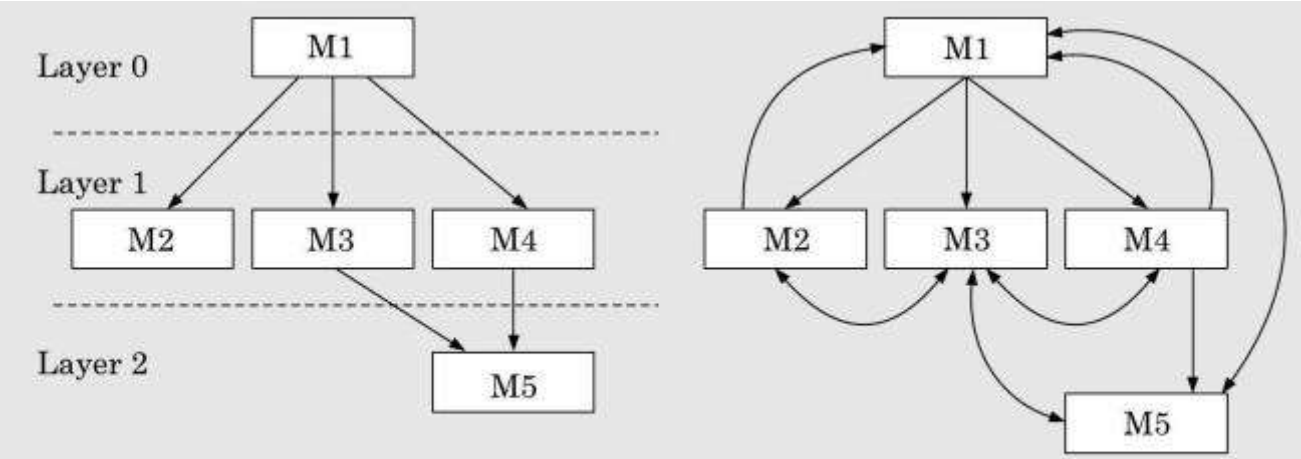
26. What are the concepts and terminologies associated with a layered design?
- Super ordinate and subordinate modules
- Visibility
- Control abstraction
- Depth and width
- Fan-out
- Fan-in

26. What do you meant by super ordinate and subordinate modules
In a control hierarchy, a module that controls another module is said to be superordinate to it. Conversely, a module controlled by another module is said to be subordinate to the controller.

27. When is module called visible?
A module B is said to be visible to another module A, if A directly calls B. Thus only the immediately lower layer modules are said to be visible to a module.

28. What is control abstraction?
The modules at the higher layers should not be visible to the modules at the lower layers. This is referred to as control abstraction.

29. What is Fan-out?
Fan-out is a measure of the number of modules that are directly controlled by a given module.



- In fig, the fan-out of module M1 is 3.

***30.* Define Fan-in?**

- Fan-in indicates the number of modules that directly invoke a given module.
- The fan-in of the module M1 is 0, that of M2 is 1, and that of M5 is 2.

***31.* What are the salient features of the function-oriented design approach?**

- Top-down decomposition
- Centralized system state

***32.* What is top-down decomposition?**

- In top-down decomposition, starting at a high-level view of the system, each high-level function is successively refined into more detailed functions
- Ex. A function **create-new-library member** is refined into the following subfunctions: o Assign-membership-number
  - o Create-member-record
  - o Print-bill

***33.* Define system state?**

- The system state can be defined as the values of certain data items that determine the response of the system to a user action or external event.
- Ex. The set of books (ie. whether borrowed by different users or available for issue) determines the state of a library automation system.
- The system state is **centralized and shared among different functions.** Eg, the libaray management system functions share data **member-records** for reference and updation.
  - o Create-new-member
  - o Create-member
  - o Update-member-record

***34.* What is object-oriented design?**

- In the object-oriented design approach, the system is viewed as a collection of objects (i.e. entities). The system state is system state is decentralized among the objects and each object manages its own state information.

***35.* What is data abstraction?**

Any entity external to the object have no knowledge about how data is exactly stored, organized, and manipulated inside the object. The entities external to the object can access the data internal to an object only by calling certain well-defined methods supported by the object.

36. What are the advantages of developing an application using ADTs?

The data of objects are encapsulated within the methods. No program element is allowed to change data except through invocation of one of the methods

- An ADT based design displays high cohesion and low coupling.
- Since abstraction used, it makes the design solution easily understandable and helps to manage complexity.

37. Write Object-oriented Vs Function-oriented design approaches?

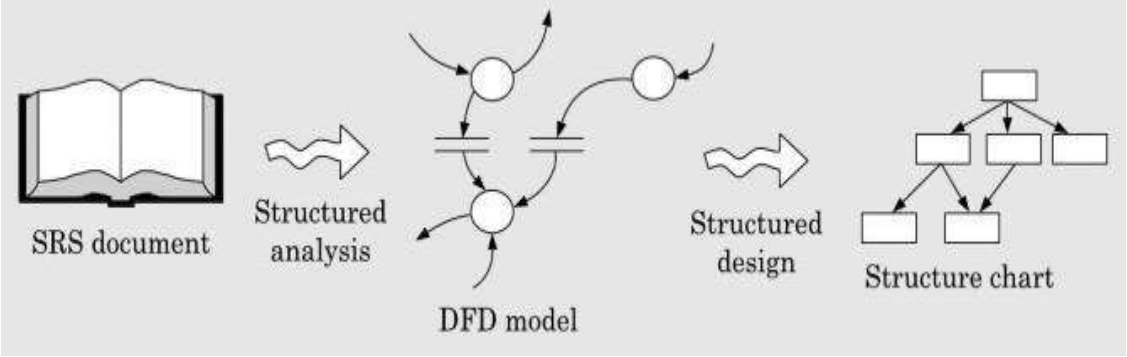
SNO	Object-oriented design approach	Function-oriented design approach
1.	The state information distributed among several objects of the system	The state information in a centralized shared data store.
2.	It group functions together on the basis of the data they operate on.	It group functions together that constitute a high level function
3.	The basic unit of designing object- oriented programs is objects. Objects appear as nouns	The basic unit of procedural design is functions. Function appear as verbs.

38. What is top-down decomposition?

The top-down decomposition used to denote the successive decomposition of a set of high- level functions into more detailed functions.

39. What is the role of Structured analysis(SA) and Structured design(SD)?

- During structured analysis, the SRS document is transformed into a DFD (Data Flow Diagram) model, functional decomposition of the system is achieved and **capture the detailed structure of the system as perceived by the user.**
- During structured design, the DFD model is transformed into a structure chart and all functions identified during structured analysis are mapped **to a module structure that is suitable for implementation in some programming language.**



40. What is detailed design?

During the detailed design stage, the algorithms and data structures for the individual modules are designed. The detailed design can directly be implemented as a working system using a conventional programming language.

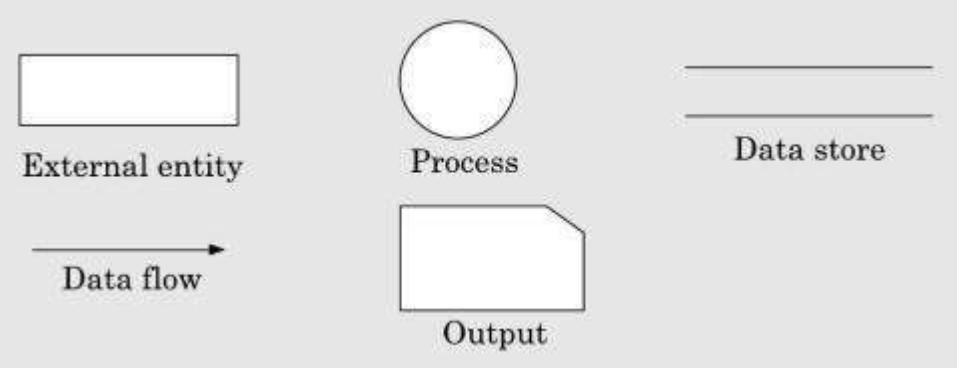
41. What are the principles followed in structured analysis technique?

- Top-down decomposition approach
- Application of divide and conquer principle. Through this, each high-level function is independently decomposed into detailed functions.
- Graphical representation of the analysis results using Data Flow Diagrams (DFDs).

42. Define Data Flow Diagram(DFD) or bubble chart?

A DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among those functions.

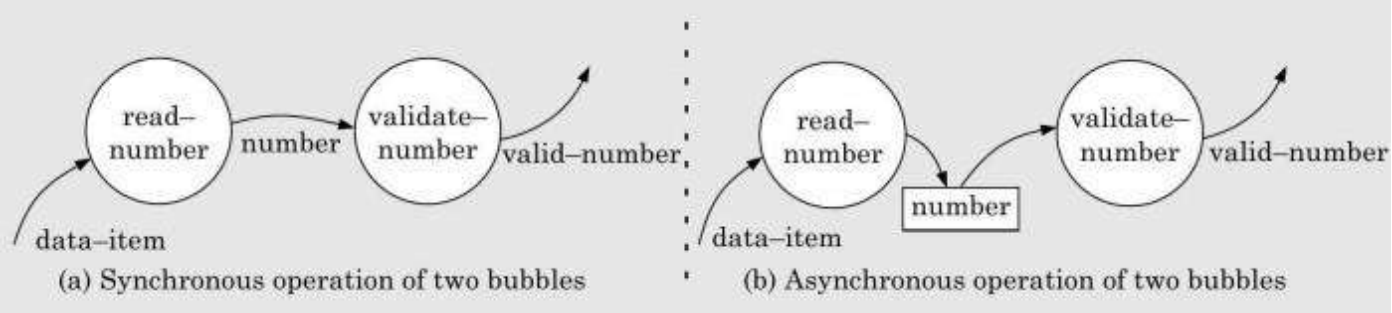
43. What are the primitive symbols used for constructing DFDs?



- Function (process or bubble) symbol – A function is represented using a circle.
- External entity symbol – Physical entities which interact with the system by inputting data to the system or consuming data produced by the system. This is represented by rectangle.
- Data flow symbol - The data flow occurring between two processes or between an external entity and a process in the direction of the data flow arrow. It is represented by di rected arc(arrow).
- Data store symbol – It is referred to either a data structure or a physical file on disk. It is represented by two parallel lines.
- Output symbol – the output symbol is used when hard copy is produced.

44. What is synchronous and asynchronous operations?

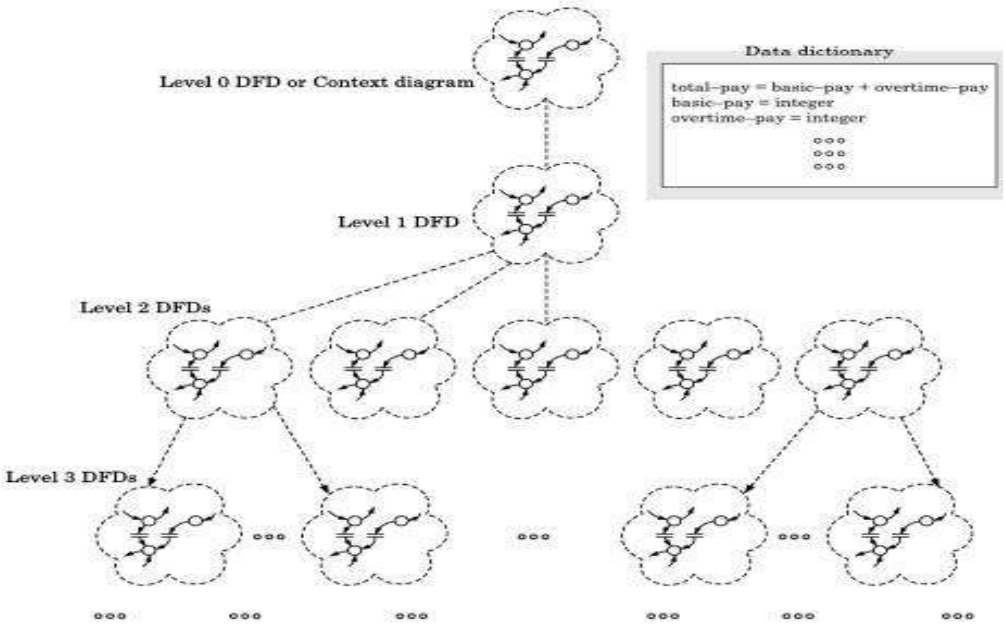
- If two bubbles are directly connected by a data flow arrow, then they are synchronous. It means that they operate at the same speed.
- If two bubbles are connected through a data store then the speed of operation of the bubbles are i ndependent called asynchronous operation.



45. Define Data dictionary?

A data dictionary lists the purpose of all data items and the definition of all composite data items in terms of their component data items.

46. Draw DFD model?



47. What is context diagram (Level0 DFD)?

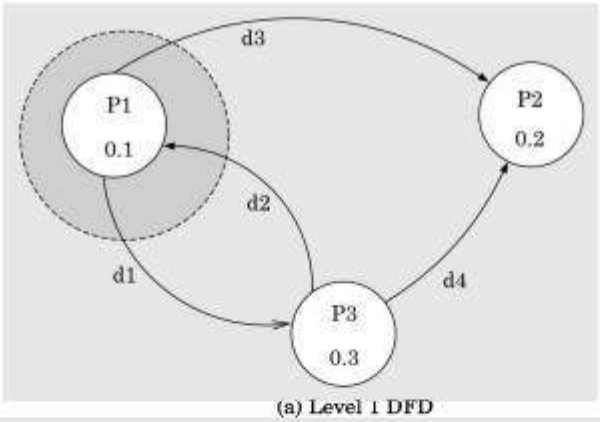
The context diagram establishes the context in which the system operates. That is who are users, what data do they input to the system, and what data they received by the system.

48. What is factoring or exploding?

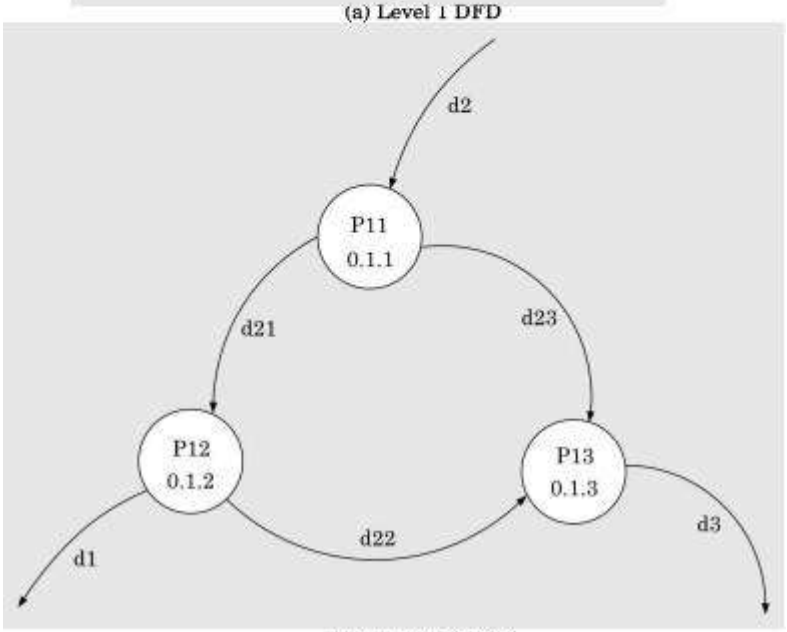
- The bubbles are decomposed into sub functions at the successive levels of the DFD model. Decomposition of a bubble is also known as factoring or exploding a bubble.
- Each bubbles at any level of DFD is decomposed to anything between 3 or 7 bubbles.

49. What do you mean by a Balancing DFD's?

The data that flow into or out of a bubble must watch the data flow at the next level of DFD. This is known as balancing a DFD.



(a) Level 1 DFD



(b) A Level 2 DFD

50. How to construct Context diagram?

The SRS document is examined to determine:

- Different high-level function that the system needs to perform
- Data input to every high-level function
- Data output from every high-level function
- Interactions(data flow) among the identified high-level functions

These aspects of the high-level function are then represented in a diagrammatic form. The form the top-level data flow diagram(DFD), usually called the DFD 0.

51. How to construct level 1 DFD diagram?

The high-level function described in the SRS document are examined. If there are between three to seven high-level requirements in the SRS document, then each of the high-level

function can be represented in the form of a bubble. If there are more than seven bubbles, then some of them have to be combined. If there are less than three bubbles, then some of these have to be split

52. How to construct lower level diagrams?

1. Each high-level function is decomposed into its constituent sub-function through the following set of activities:

    - Different sub-function of the high-function are identified
    - Data input to each of these sub-function is identified.
    - Data output from each of these sub-function are identified
    - Interactions(data flow) among these sub-function are identified These

    aspects are then represented in a diagrammatic form DFD.

2. Step 1 is repeated recursively for each sub-function until a sub-function can be represented by using a simple algorithm.

53. How do you number bubbles in DFDs?

- It is necessary to number the different bubbles occurring in the DFD to uniquely identify all bubble in the DFD. The bubble at the context level is usually assigned the number 0 to indicate that it is the 0 level DFD. Bubbles at level 1 are numbered 0.1, 0.2, 0.3, etc.
- When a bubble numbered x is decomposed, its children bubble are numbered x1, x2, x3, etc. in this numbering scheme, by looking at the number of the bubble we can unambiguously determine its level, its ancestors and its successors.

54. What are the errors commonly made while constructing a DFD model?

- Many beginners commit the mistakes of drawing more than one bubble in the context diagram. The context diagram should depict the system as a single bubble.
- Many beginners have external entities appearing all levels of DFDs. All external entities interacting with the system should be represented only in the context diagram. The external entities should not appear at other levels of the DFD.
- It is a common oversight to have either too less or too may bubbles in a DFD. Only three to seven bubbles per diagram should be allowed, i.e. each bubble should be decomposed to between three and seven bubbles.
- Many beginners leave different levels of DFD unbalanced.

A common mistake committed by many beginners while developing a DFD model is

attempting to represent control information in a DFD. It is important to realize that a DFD is the data flow representation of a system and it does not represent control information.

55. What are the shortcomings of the DFD model?

- DFDs leave ample scope to be impreceise
- Control aspects are not defined by a DFD
- Decompostion is carried out are highly subjective and dependon the choice and judgment of the analysist.
- Data flow diagram does not provide any specific guidance as to how exactly to decompose a given function into its subfunctions.

56. What is control specification (CSPEC)?

The CSPEC describes the following:

- The effect of an external event or control signal
- The process that are invoked as a consequence of an event

Control specification represent the behaviour of the system in two different ways:

- It contains a state transition diagram (STD). The STD is a sequential specification of behaviour.
- It contains a program activation table (PAT). The PAT is a combinatorial specification of behaviour. PAT represents invocation sequence of bubbles in a DFD.

57. Define structure chart?

A structure chart represents the software architecture that is the various modules making up the system, the module dependency (i.e. which module calls which other modules), and
the parameters that are passed among the different modules), and the parameters that are passed among the different modules

58. What are basic building blocks of structure chart?

- **Rectangular boxes**. A rectangular box represents a module. Usually. Every rectangular box is annotated with the name of the module it represents.
- **Module invocation arrows**. An arrow connecting two modules implies that during program execution, control is passed from one module to the other in the direction of

connecting arrows. However, just by looking at the structure chart, we cannot say whether a module calls another module just once or many times. Also, just by looking at the structure chart, we cannot tell the order in which the different modules are invoked.

- **Data flow arrows**. These are small arrows appearing alongside the module invocation arrows. The data flow arrows are annotated with the corresponding data name. The data flow arrows represent the fact that the named data passes from one module to the other in the direction of the arrow.

- **Library modules**. A library module is usually represented by a rectangle with double edges. Libraries comprise the frequently called modules. Usually, when a module is invoked by many other modules, it is made into library module.

- **Selection**. The diamond symbol represents the facts that one module out of several modules connected with the diamond symbol is invoked depending on the outcome of the condition attached to the diamond symbol.

- **Repetition**. A loop around the control flow arrows denotes that the respective modules are invoked repeatedly.

59. Write Flow Chart versus Structure Chart?

A structure chart differs from a flow chart in three principal way:

- It is usually difficult to identify different modules of the software from its flow chart representation.
- Data interchange among different modules is not represented in s flow chart.
- Sequential ordering of tasks inherent in a flow chart is suppressed in a structure chart.

60. What are the strategies involved in structured design?

- Structured design provides two strategies to guide transformation of a DFD into a structured chart:
    - **Transform analysis**
    - **Transaction analysis**

Normally, one would start with the level 1 DFD, transform it into module representation using either the transform or transaction analysis and then proceed toward the lower level DFDs.

61. Write about transform analysis?

- Transform analysis identifies the primary functional components (modules) and the high level input and outputs for these components.

The **first step** in transform analysis is to divide the DFD into three types of parts: **o Input**

- **o Logical processing**
- **o Output**
- o The input portion in the DFD includes processes that transforms input data from physical (e.g. character from terminal) to logical form (e.g. internal tables, lists, etc.). Each input portion is called an afferent branch.
- o The output portion of a DFD transforms output data from logical form to physical form. Each output portion is called an efferent branch. The remaining portion of a DFD is called central transform.

In the **next step** of the transform analysis the structure chart is derived by drawing one functional component for the central transform, and the afferent and efferent branches. These are drawn below a root module, which would invoke these modules.

Identifying the highest level input and output transforms requires experience and skill. One possible approach is to trace the inputs until a bubble is found whose output cannot be deduced from its inputs alone.

Processes which validate input or add information to them are not central transforms. Processes which sort input or filer data from it are central transform. The first level of structure chart is produced by representing each input and output unit as boxes and each central transform as a single box.

In the **third step** of transform analysis, the structure chart is refined by adding sub- functions required by each of the high level functional components. Many levels of functional components may be added. This process of breaking functional components into subcomponents is called factoring.

Factoring includes adding read and write modules, error-handling modules, initialization and termination processes, identifying consumer modules, etc. The factoring process is continued until all bubbles in the DFD are represented in the structure chart.

61. Write about transaction analysis?

- Transaction analysis is an alternative to transform analysis and is useful while designing transaction processing programs
- A transaction allows the user to perform some meaningful piece of work by using the software.
- In a transaction-driven system, different data items may pass through different computation paths through the DFD.

62. What is detailed design?

During detailed design the pseudo code description of the processing and the different data structures are designed for the different modules of the structure chart. These are described in the form of **module specification (MSPEC).** It is written using structured English.

63. What are the aspects considered in design review?

After a design is complete, the design is required to be reviewed.
The review team checks the design documents especially for the following aspects: o Traceability

- Correctness
- Maintainability
- Implementation

64. List the principles of software design (Nov 2017)

1.Modularization 2. Abstraction 3. Encapsulation 4. Coupling 5. Cohesion 6. Separation of interface and implementation 7. Sufficiency 8. Completeness

11 marks

1. Explain Different approaches to Software Design?

Design Approaches

- Two fundamentally different software design approaches: o Function-oriented design
  - Object-oriented design
- These two design approaches are radically different. o However, are complementary
  - rather than competing techniques. o Each technique is applicable at
  - different stages of the design process. **Function-Oriented Design**
- A system is looked upon as something o that performs a set of functions.
- Starting at this high-level view of the system:
  - each function is successively refined into more detailed functions.
  - Functions are mapped to a module structure.
- Example
  The function **create-new-library** member:
  - creates the record for a new member,
  - assigns a unique membership number

- prints a bill towards the membership

Create-library-member functionconsistsofthefollowingsub-functions: o assi gn-membership-number
- create-member-record
- print-bill

Function-Ori ented Desi gn
- Each subfunction is split into more detailed subfunctions and so on.

- The **system state** is centralized:
  - accessible to different functions,
  - member-records:
    - available for reference and updation to several functions:
      - create-new-member
      - delete-member
      - update-member-record

Object-ori ented desi gn

- System is viewed as a collection of objects (i.e. entities).
- System state is decentralized amongthe objects:
  - each object manages its own state information.
- Object-OrientedDesignExample Li brary Automation Software:
  - each library member is a separate object
    - with itsown dataandfunctions. o

    Functions defined for one object:
    - cannot directly refer to or change data of other objects.
- Objects have their own internal data: o defines their state.
- Similar objects constitute a class.
  - each object i s a member of some class.
- Classes may inherit features o from a super class.
- Conceptually, objects communi cate by message passing. **Object-Ori ented versus Functi on-Ori ented Desi gn**

- Unlike function-oriented design,
  - in OOD the basic abstraction is not functions such as "sort", "display", "track", etc.,
  - but real-world entities such as "employee", "picture", "machine", "radar system", etc.
- I n OOD:
  - software is not developed by designing functions such as:
    - update-employee-record,
    - get-em ployee-address, etc. o

    but bydesigningobjects such as:
    - employees,
    - departments, etc.
- Grady Booch sums up this fundamental difference saying:
  - "Identify verbs if you are after procedural design and nouns if you are after object-oriented design."
- I n OOD:
  - state information is not shared in a centralized data.

- but is distributed among the objects of the system.

- Example:
  In an employee **pay-roll system**, the following can be global data:
  - names of the employees,
  - their code numbers,
  - basic salaries, etc.

  Whereas, in object oriented systems:
  - data is distributed among different employee objects of the system.
- Objects communicate by message passing.
  - one object may discover the state information of another object by interrogating it.
- Of course, somewhere or other the functions must be implemented:
  - the functions are usually associated with specific real-world entities (objects)
  - directly access only part of the system state information.
- Function-oriented techniques group functions together if:
  - as a group, they constitute a higher level function.
- On the other hand, object-oriented techniques group functions together:
  - on the basis of the data they operate on.
- To illustrate the differences between object-oriented and function-oriented design approaches,
  - let us consider an example---
  - An automated fire-alarm system for a large building.

Fire-Alarm System:

- We need to develop a computerized fire alarm system for a large multi-storied building:
  - There are 80 floors and 1000 rooms in the building.
- Different rooms of the building:
  - fitted with smoke detectors and fire alarms.
- The fire alarm system would monitor:
  - status of the smoke detectors.
- Whenever a fire condition is reported by any smoke detector:
  - the fire alarm system should:
    - determine the location from which the fire condition was reported
    - sound the alarms in the neighboring locations.
- The fire alarm system should:
  - flash an alarm message on the computer console:
    - fire fighting personnel man the console round the clock.
- After a fire condition has been successfully handled,
  - the fire alarm system should let fire fighting personnel reset the alarms.

Function-Oriented Approach:

```
/* Global data (system state) accessible by various functions */ BOOL
                detector_status[1000];
int             detector_locs[1000];
BOOL                    alarm-status[1000]; /* alarm activated when status set */ int
                alarm_locs[1000]; /* room number where alarm is located */
int              neighbor-alarms[1000][10]; /*each detector has at most*/
                  /* 10 neighboring alarm locations */
```

The functions which operate on the system state:

interrogate_detectors();

get_ detector_locati on();
determine_neighbor_alarm(); determi
ne_neighbor_sprinkler(); ring_alarm();
acti vate_spri nkler();
reset_alarm();
reset_spr i nkler();
report_fi re_locati on();

Object-Oriented Approach:

class detector
attri butes: status, locati on, nei ghbors
operations: create, sense-status, get-location, find-neighbors **class** alarm
attri butes: locati on, status
operations: create, ring-alarm, get_location, reset-alarm
class sprinkler
attri butes:locati ons, status
operations: create, activate-sprinkler, get_ location, reset-sprinkler

- In the object oriented program,
  - appropri ate number of i nstances of the class detector and alarm should be created.
- In the function-oriented program :
  - the system state is centralized
  - several functions accessing these data are defined.
- In the object oriented program,
  - the state information is distributed among various sensor and alarm objects.
- Use OOD to design the classes:
  - then applies top-down function oriented techniques
    - to design the internal methods of classes.
- Though outwardly a system may appear to have been developed in an object oriented fashion,
  - but inside each class there is a small hierarchy of functions designed in a top- down manner.

2 Explain Cohesion and coupling

Cohesion :

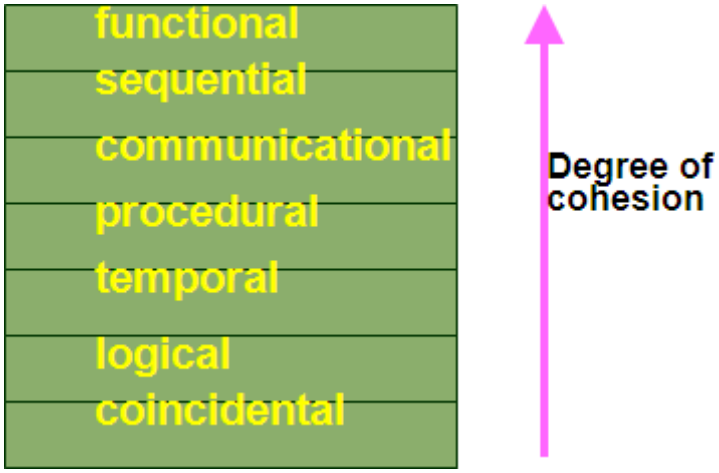
- Cohesion is a measure of:
  - functi onal strength of a module.
  - A cohesive module performs a single task or function.
- Coupling between two modules:
  - a measure of the degree of interdependence or interaction between the two modules.
- A module having high cohesion and low coupling:
  - functionally independent of other modules:
    - A functionally independent module has minimal interaction with other modules.

Advantages of Functi onal I ndependence

- Better understandability and good design:
- Complexity of design is reduced,
- Different modules easily understood in isolation: o modules are independent
- Functional independence reduces error propagation. o degree of interaction between modules is low.
  - o an error existing in one module does not directly affect other modules.
- Reuse of modules is possible.
- A functionally independent module:
  - o can be easily taken out and reused in a different program.
    - each module does some well-defined and precise function
    - the interfaces of a module with other modules is simple and minimal.

Classification of Cohesiveness

- Classification is often subjective:
  - o yet gives us some idea about cohesiveness of a module.
- By examining the type of cohesion exhibited by a module:
  - o we can roughly tell whether it displays high cohesion or low cohesion.



Coincidental cohesion

- The module performs a set of tasks:
  - o which relate to each other very loosely, if at all.
    - the module contains a random collection of functions.
    - functions have been put in the module out of pure coincidence without any thought or design.

Logical cohesion

- All elements of the module perform similar operations: o e.g. error handling, data input, data output, etc.
- An example of logical cohesion:
  - o a set of print functions to generate an output report arranged into a single module.

Temporal cohesion

- The module contains tasks that are related by the fact:
  - o all the tasks must be executed in the same time span.
- Example:
  - o The set of functions responsible for

- initialization,
- start-up, shut-down of someprocess, etc.

Procedural cohesion

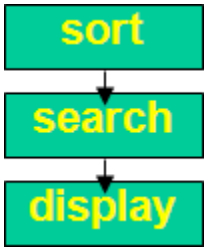
- The set of functions of the module:
  - all part of a procedure (algorithm)
  - certain sequence of steps have to be carried out in a certain order for achieving an objective,
    - e.g. the algorithm for decoding a message.

Communicational cohesion

- All functions of the module:
  - reference or update the same data structure,
- Example:
  - the set of functions defined on an array or a stack.

Sequential cohesion

- Elements of a module form different parts of a sequence,
  - output from one element of the sequence is input to the next.
  - Example:



Functional cohesion

- Different elements of a module cooperate: o to achieve a single function,
  - e.g. managing an employee's pay-roll.
- When a module displays functional cohesion,
  - we can describe the function using a single sentence.
- Determining Cohesiveness
- Write down a sentence to describe the function of the module o If the sentence is compound,
    - it has a sequential or communicational cohesion. o If it has words like "first", "next", "after", "then", etc.
    - it has sequential or temporal cohesion. o If it has words like initialize,
    - it probably has temporal cohesion.

Coupling

- Coupling indicates:
  - how closely two modules interact or how interdependent they are.
  - The degree of coupling between two modules depends on their interface complexity.
- There are no ways to precisely determine coupling between two modules:
  - classification of different types of coupling will help us to approximately estimate the degree of coupling between two modules.
- Five types of coupling can exist between any two modules.
- Classes of coupling

Data coupling

- Two modules are data coupled,
  - if they communicate via a parameter:
    - an elementary data item,
    - e.gan integer, a float, a character, etc. o

    The data item should be problem related:
    - not used for control purpose.

Stamp coupli ng

- Two modules are stamp coupled,
  - if they communicate via a composite data item
    - such as a record in PASCAL
    - or a structure in C.

Control coupling

- Data from one module is used to direct
  - order of instruction execution in another.
- Example of control coupling:
  - a flag set in one module and tested in another module.

Common Coupli ng

- Two modules are common coupled,
  - if they share some global data.

Content coupli ng

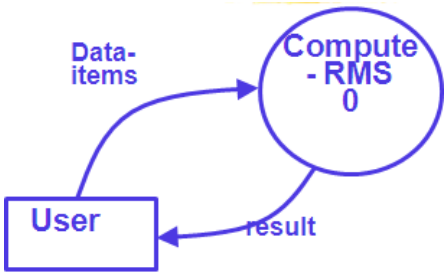
- Content coupling exists between two modules: o i f they share code,
  - e.g, branching from one module into another module.

- The degree of coupling increases from data coupling to content coupling.

3. Design RMS calculati ngsoftware?

- Consider a software called RMS calculating software:
  - reads three integers in the range of -1000 and +1000
  - o finds out the root mean square (rms) of the three input numbers displays the result

- **Context di agram (Level 0 DFD diagram)**
  The context di agram i s si mple to develop:
  - The system accepts 3 integers from the user
  - returns the result to him.



Context Diagram

- **From a cursory analysis of the problem description:**

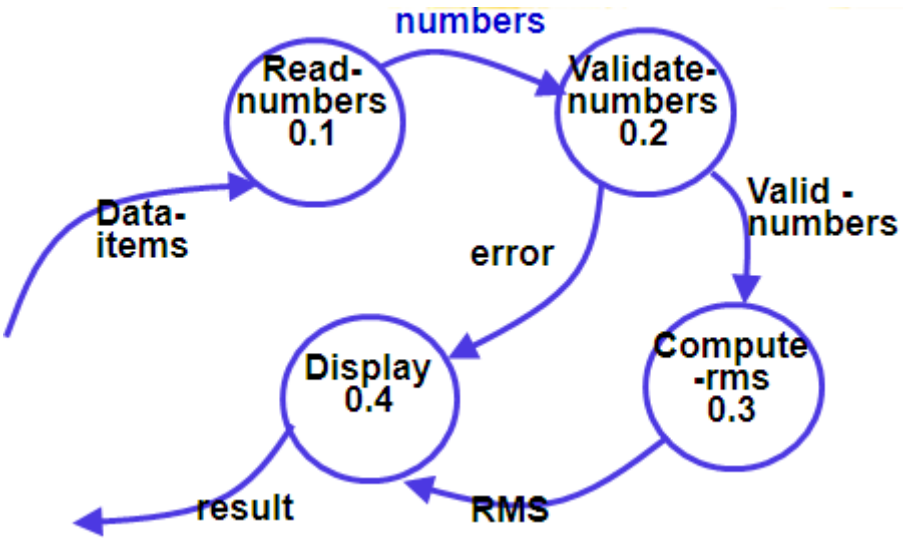
  we can see that the system needs to perform several things. o A

  ccept input numbers from the user:

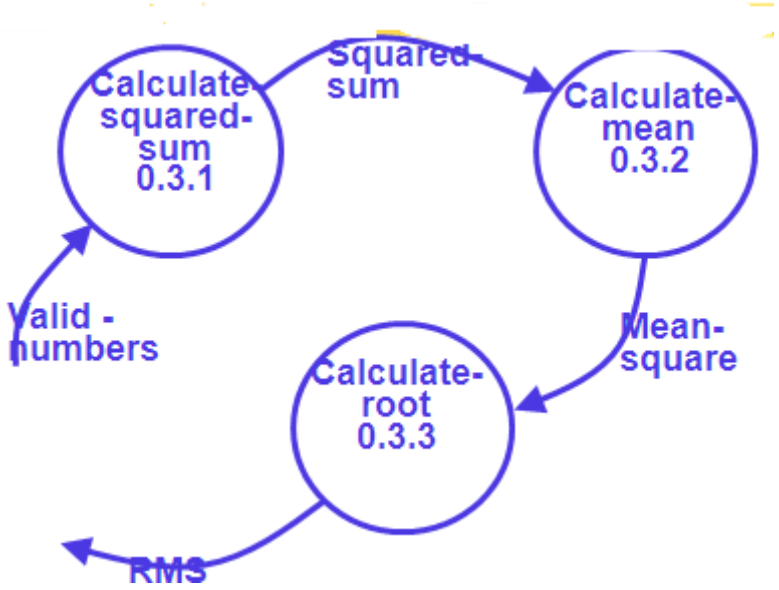
  - o validate the numbers,
  - o calculate the root mean square of the input numbers display the

  result

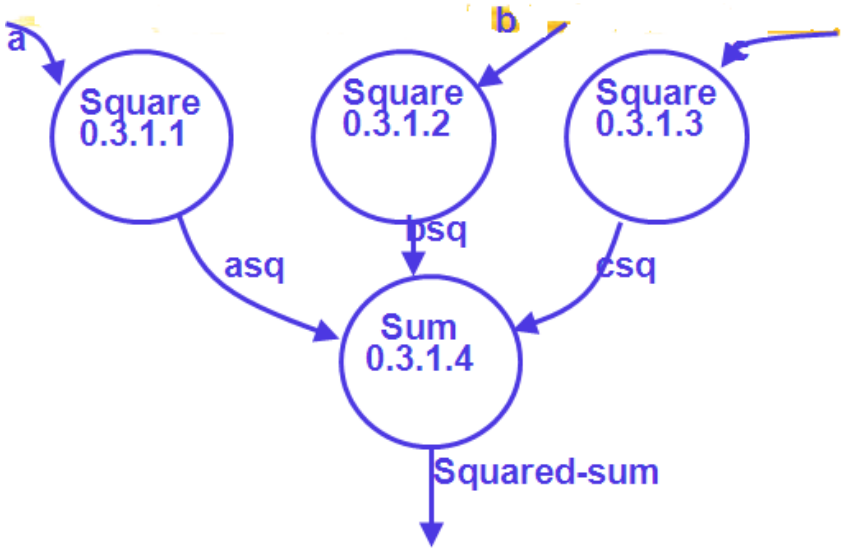
- **Level 1 DFD diagram**



- **Level 2 DFD diagram**



- **Level 3 DFD diagram**



- **Data dictionary for RMS Software**

numbers=vali d-numbers=a+ b+ c a:i

nteger * input number *

b:integer * input number *

c:integer * input number *

asq:integer

bsq:i nteger

csq:i nteger

squared-sum: i nteger R

esult=[RMS,error]

R MS: integer * root mean squarevalue* error

:string * er ror message*

- **Structure chart**

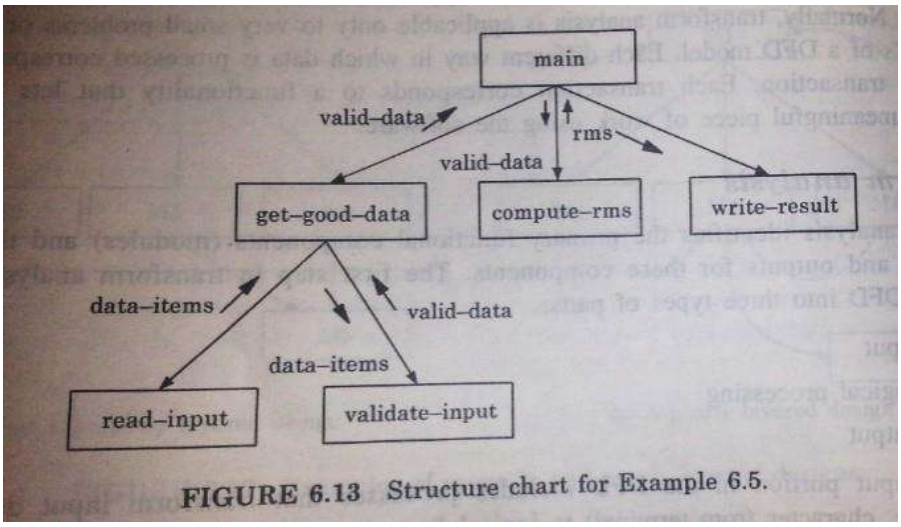
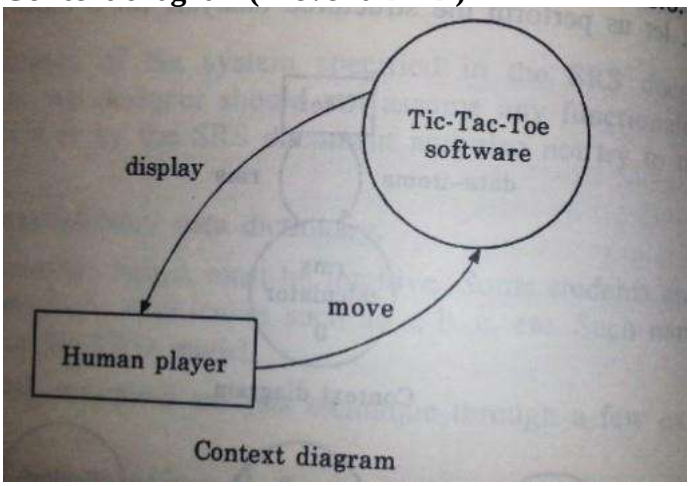


FIGURE 6.13 Structure chart for Example 6.5.

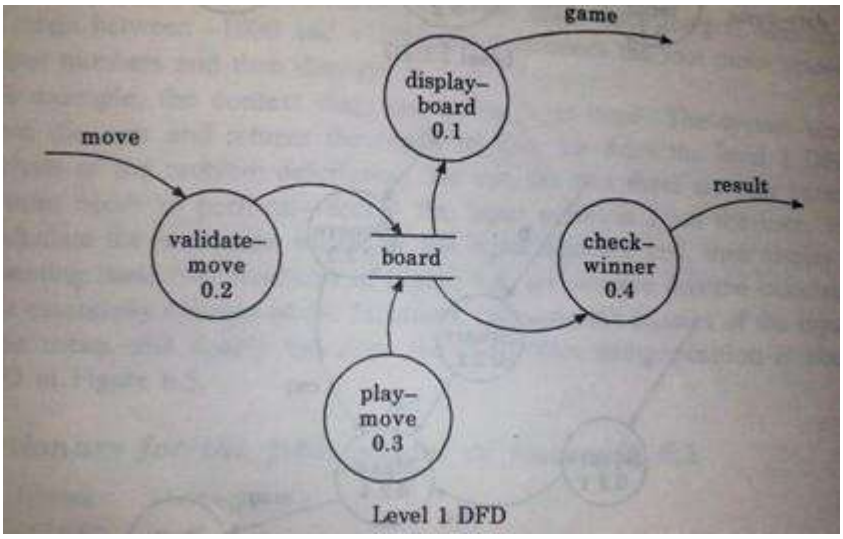
4 Design Tic-Tac-Toe computer game?

Tic-Tac-Toe is a computer game in which a human player and the computer make alternate moves on a 3x3 square. A move consists of marking a previously unmarked square. The player who is first to place three consecutive marks along a straight line (i.e. along a row, column, diagonal on the square wins. As soon as either the human player or the computer wins, a message congratulating the winner is displayed. If neither player manages to get three consecutive marks along a straight line, and all the squares on the board are filled up, then the game is drawn. The computer always tries to win a game.

- **Context diagram(Level 0 DFD)**



- **Level 1DFD diagram**



Level 1 DFD

- **Data dictionary**

  Di splay:gam e + result

  Game:board

  Board:(i nteger)9

  R esult:[“ computer won”,”human won”,”drawn”]

- **Structure chart**

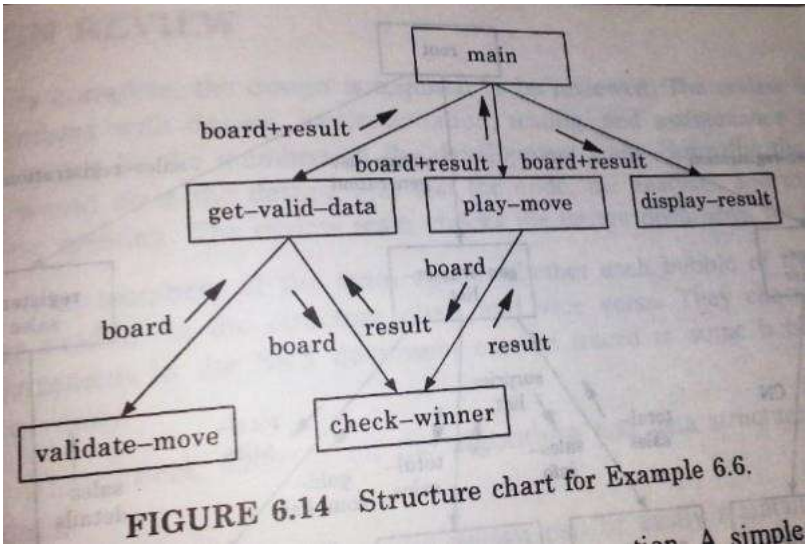


FIGURE 6.14 Structure chart for Example 6.6.

5. Desi gn Supermarket Pr i ze Scheme software?

A supermarket needs to develop the following software to encourage regular customers. For this, the customer needs to supply his residence address, telephone number, and the driving license number. Each customer who registers for this scheme is assigned a unique customer number (CN) by the computer. A customer can present his CN to the check-out staff when he makes any purchase. In thi s case, the value of hi s purchase i s credi ted agai nst hi s CN. A t the end of each year , the supermarket awards surpri se gifts to 10 customers who make the hi ghest total purchase over the year. A lso ,i t awards a 22 carat gold coi ns to every customer who purchase exceed R s.10,000.T he

entries against the CN are reset on the last day of every year after the prize winners list are generated.

- **Context diagram(Level 0 DFD)**

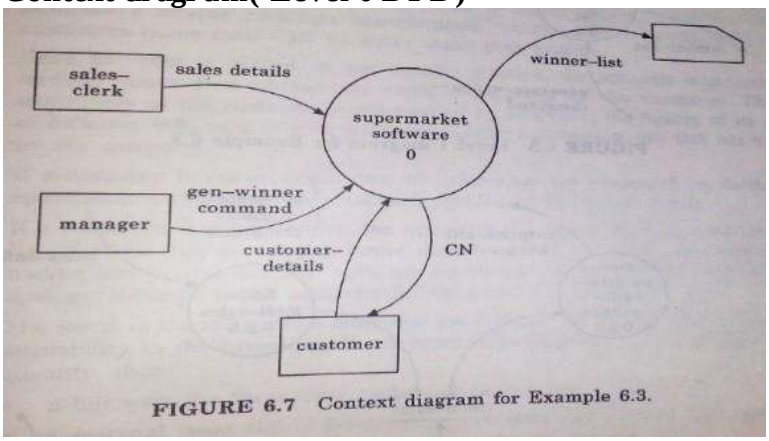


FIGURE 6.7 Context diagram for Example 6.3.

- **Level 1DFD diagram**

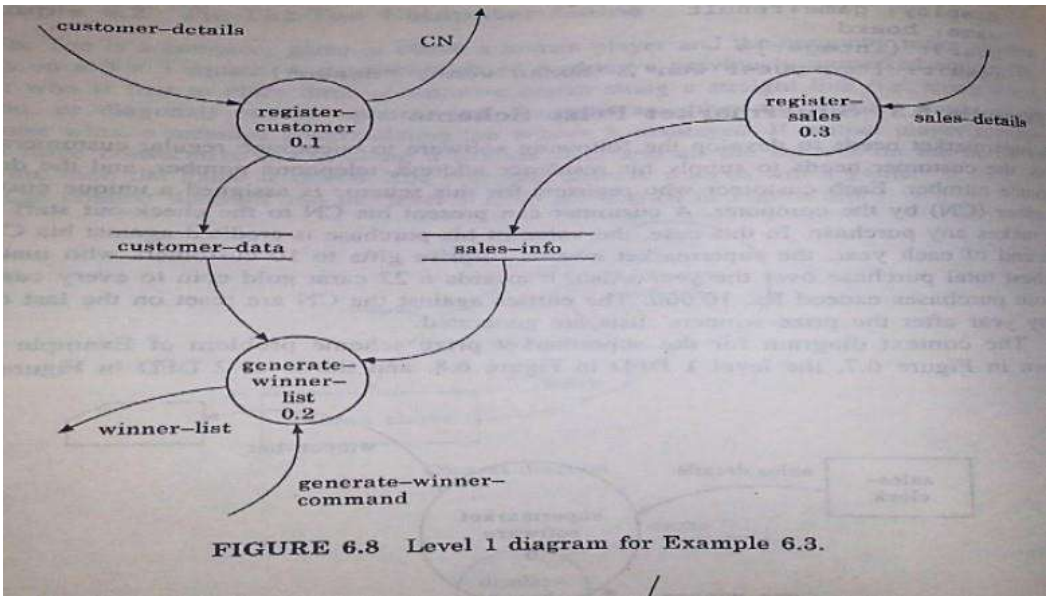


FIGURE 6.8 Level 1 diagram for Example 6.3.

- **Level 2 DFD diagram**

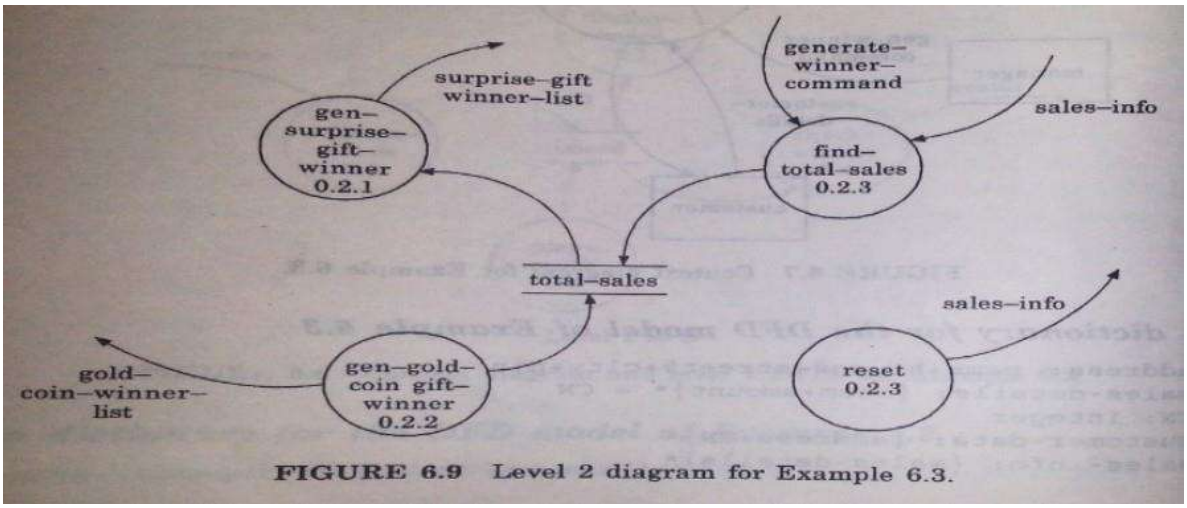


FIGURE 6.9 Level 2 diagram for Example 6.3.

- **Data Dictionary for the DFD model**

  Address: name +house#+street#+city+pin Sales-

  details:(item+amount)*+CN

  CN :integer

  Customer-data:{address +CN}*

  Sales- i nfo:{sales-detai ls}*

  Winner-list:surprise-gift-winners-list+gold-coin-winner-list Surprise-

  gift-winner-list:(address +CN)*

  Gold-coi n-winner-list:{address +CN}*

  Gold –winner-command:command Total-

  sales:{CN+i nteger }*

- **Structure chart**

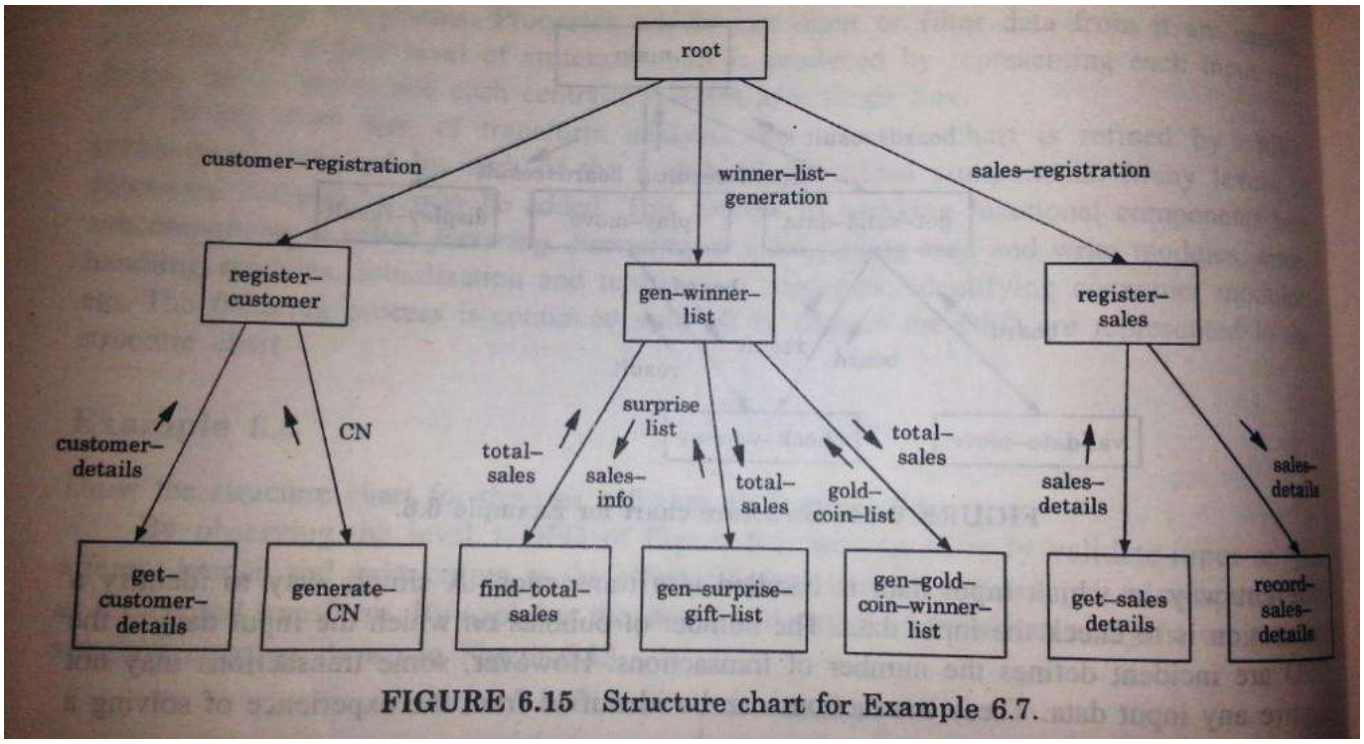


FIGURE 6.15 Structure chart for Example 6.7.

6. Design Tradinghouse automation software?

A trading house wants us to develop a computerized system that would automate various bookkeepingactivities associated with its business. The followingare the silent features the system to be developed:

- The trading house has a set of regular customers .The customers of orders with it various kinds of commodities .The tradition house maintains the names with and address of its regular customers .Each of these regular customers should be assigneda unique customer identification number (CIN) by the computer. The customers quote their CN on every order they place.

- Once an order is placed, as per current practice, the accounts department of the trading house first checks the credit – worthiness of the customer. The credit worthiness of the customer is determined by analysis the history of its payments to different bills sent to him in the past. After automation, this task has to be done by the computer.

- If a customer is not credit-worthy, his orders are not processed any further and an appropriate order rejection message is generated for the customer.

- If a customer is credit worthy, the items that he has ordered has checked against a list of items that the trading house deals with. The items in the order which the trading house does not deal with, are not processed any further and an appropriate

apology message to the customer for these i ts i s gener ated.

- The items in the customers order that the trading house deals with are checked for available in the i nventory. If the items are available in the inventory in desired quantity ,then

  - ✓ A bill with the forwarding address of the customer is printed.
  - ✓ A material issue slip is printed. The customer can produce the material issue slip at the storehouse and take delivery of the items.
  - ✓ Inventory data is adjusted to reflect the sale to the customer.

- If any of the ordered items are not available in the inventory in sufficient quantity to satisfy the order ,then these out of stock items along with the quantity ordered by the customer and its CIN are stored in a "pending order" file for further processing to be carried out when the purchase department issues the " generate indent" command.
- The purchase department should be allowed to peri odically issue commands to generate indents. When a command to generate indent is issued, the system should be examine the "pending order" file to determi ne the order that are pending and to determine the quantity required for each of the items. It should find out the addresses of the vendors who supply these items by exami ni ng the fi le contai ni ng vendor details and them should print out indents to those vendors.
- The system should also answer managerial queries regarding the statistics of different items sold over any given period of time and the corresponding quantity sold and the prize realized.

The context diagram for the trading house automation problem is shown in figure 6.10 and the level 1 DFD in figure 6.11.

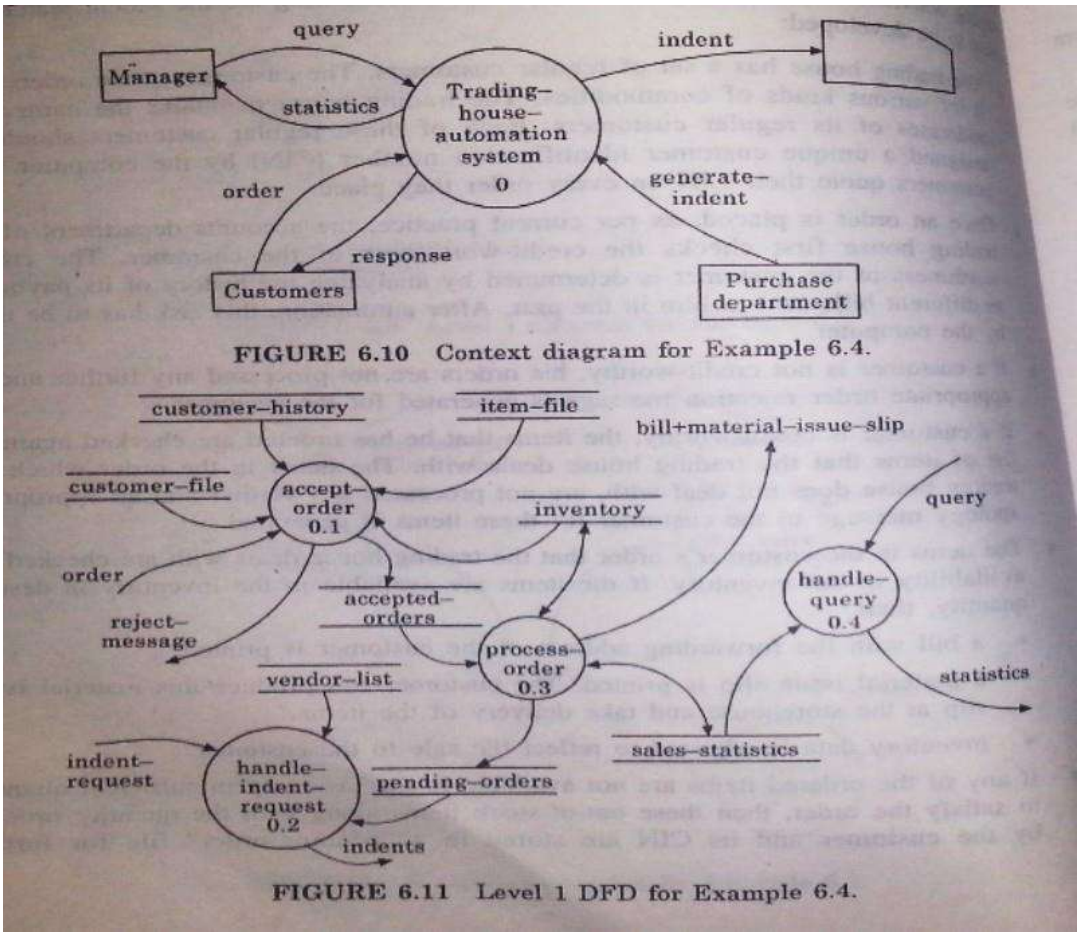


FIGURE 6.10 Context diagram for Example 6.4.

FIGURE 6.11 Level 1 DFD for Example 6.4.

Data di cti onary for the DFD model

response: [bi l l + material-i ssue-sli p, reject-message]

query: period/* query from manager regardingsales statistics*/ date: year +

month + day

year : i nteger

month: integer

day: i nteger

order: customer-i d + {i tems + quanti ty}* + order#

accepted-order: order /* ordereditems available in inventory */ reject-

message: order + message/* rejection message*/

pending-orders: customer-id+ {items+quantity}* customer-

address:name+house#+street#+city+pin name: string

house#: str i ng

street#: string ci

ty: string

pin: integer

customer-i d: i nteger

customer-fi le: {customer-address}*

bill: {item + quantity + price}* + total-amount + customer-address + order# material-i

ssue-slip: message + item + quantity + customer address

message: string

statistics: {item + quantity + price}* sales-

statistics: {statistics}*+ date quantity: i

nteger

total-amount: i nteger

gener ate-i ndent: command

indent: {item+quantity}* + vendor-address i

ndents: {indent}*

vendor-address : customer-address

vendor-list : {vendor-address}*

i tem-file: {item}* i

tem: str i ng

i ndent-request: command

Observati ons. The followi ngobservati ons can be made from Example

1. In a DFD, if two data stores deal with different types of data, e.g. one type of data is invariant with time whereas another varies with time (e.g. vendor address, and inventory data) it is a good idea to represent them as separate data stores. The inventory data changes each time fresh supply arrives or when an item is sold (and the i nventory is updated), whereas the vendor data remai nsunchanged.
2. If we are developing the DFD model of a process which is already being manually carried out, then the names of the registers being maintained in the manual process would appear as data stores in the DFD model. For example, if TAS is currently being manually carried out, then normally there would be registers corresponding to accepted orders, pending orders, vendor list, etc.
3. We can observe that DFDs enable a software engineer to develop the data domain and the functional domain models of the system at the same time. As the DFD is refined into greater levels of detail, the analyst performs an i mplicit functional decomposition. At the same time,

the DFD refinement automatically results in refinement of the corresponding data items.

Structure Chart:-

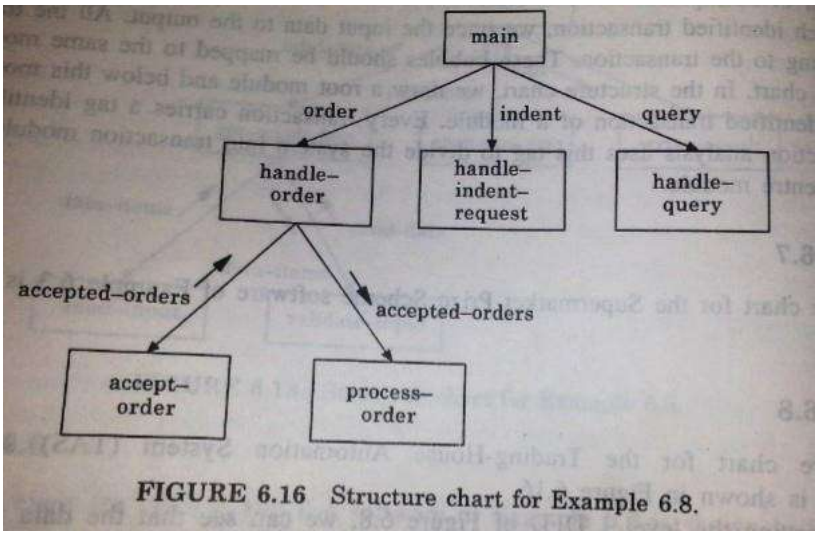
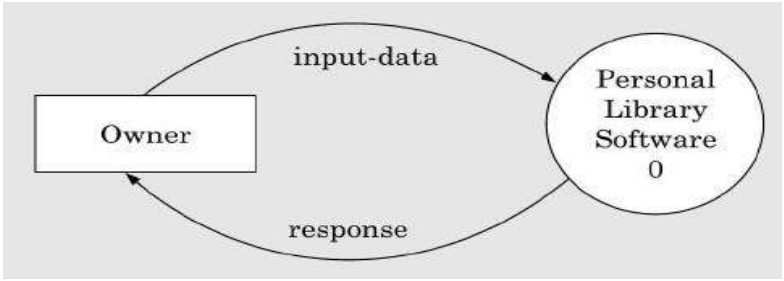


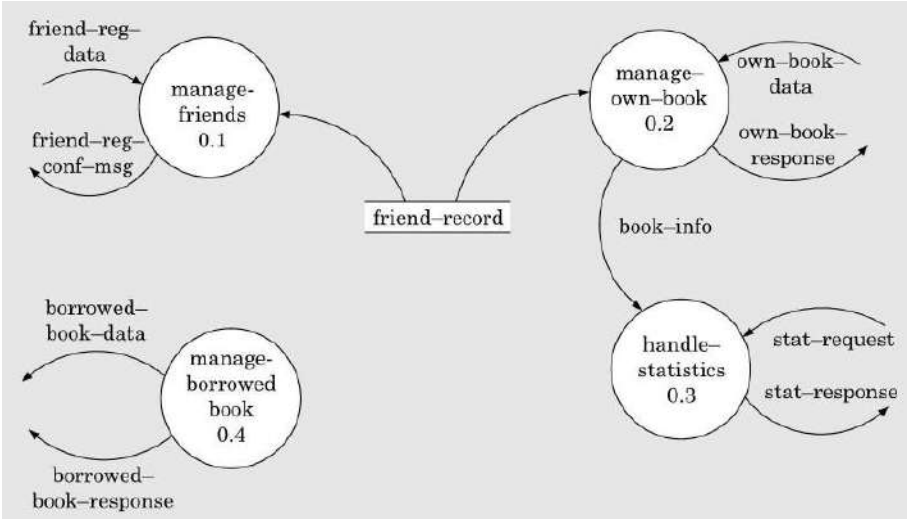
FIGURE 6.16 Structure chart for Example 6.8.

7. Design DFD and Structure chart for Personal library software?

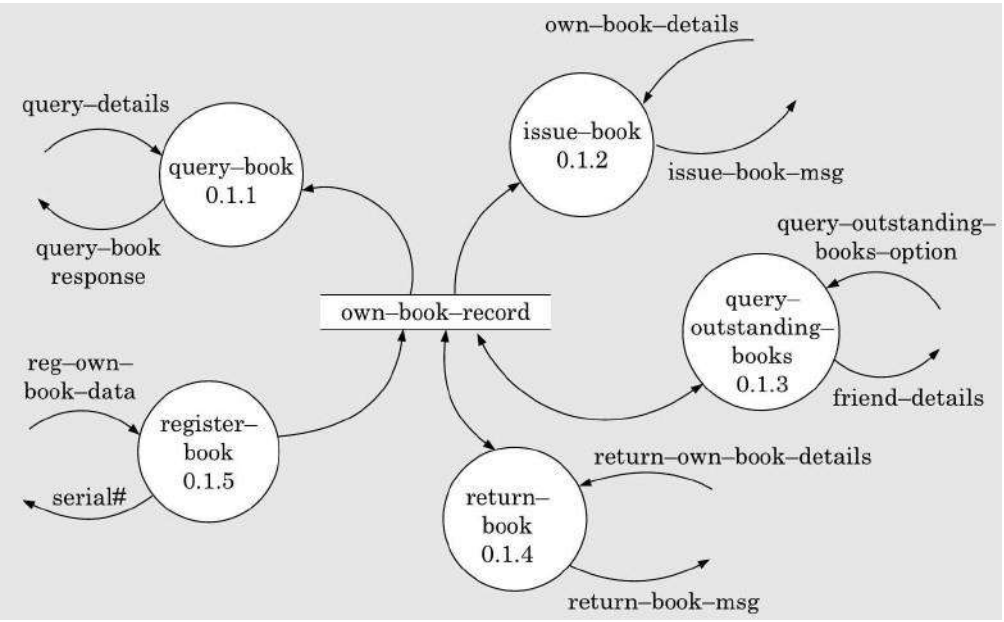
- **Level 0 DFD (context diagram)**



- **Level 1 DFD**



- **Level 2 DFD**

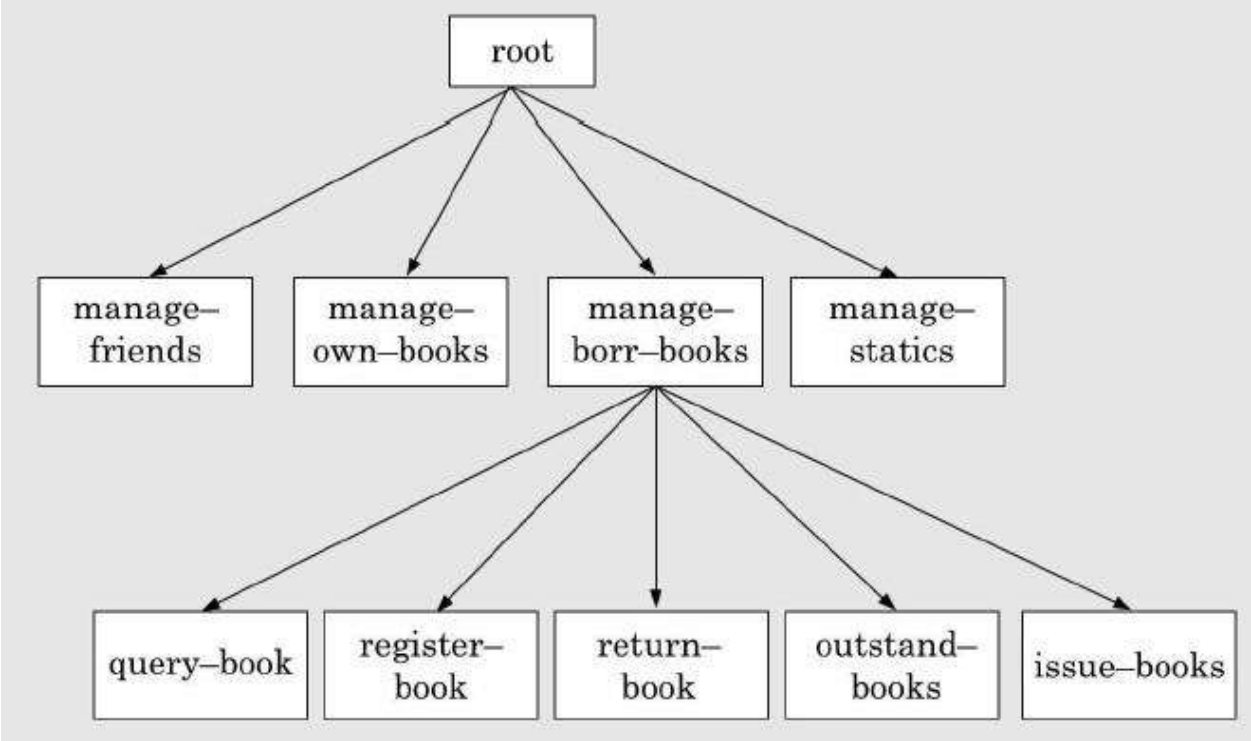


- **Data dictionary**

Data dictionary for the DFD model of Example 6.5

input-data: friend-reg-data + own-book-data + stat-request + borrowed-book-data
response: friend-reg-conf-msg + own-book-response + stat-response + borrowed-book-response
own-book-data: query-details + own-book-details + query-outstanding-books-option + return-own-book-details + reg-own-book-data
own-book-response: query-book-response + issue-book-msg + friend-details + return-book-msg + serial#.
borrowed-book-data: borrowed-book-details + book-return-details + display-books-option
borrowed-book-response: reg-msg + unreg-msg + borrowed-book-list
friend-reg-data: name + address + landline# + mobile#
own-book-details: friend-reg-data + book-title + data-of-issue
return-own-book-details: book-title + date-of-return
friend-details: name + address + landline# + mobile# + book-list
borrowed-book-details: book-title + borrow-date
serial#: integer

- **Structure Chart**



8 Explain DFD?

DATA FLOW DIAGRAMS (DFDs)

- The DFD (also known as the bubble chart) is a simple graphical formalism that can be used to represent a system in terms of the input data to the system, various [processing carried out on these data, and the output data generated by the system.
- The main reason why the DFD technique is so popular is probably because of the fact that DFD is a very simple formalism- It is simple to understand and use. A DFD model is uses a very limited number of primitive symbols (Figure 6.1) to represent the functions performed by a system and the data flow among these functions with a set of high level functions that a system performs, a DFD model hierarchically represents various sub-functions. In fact, any hierarchical model is

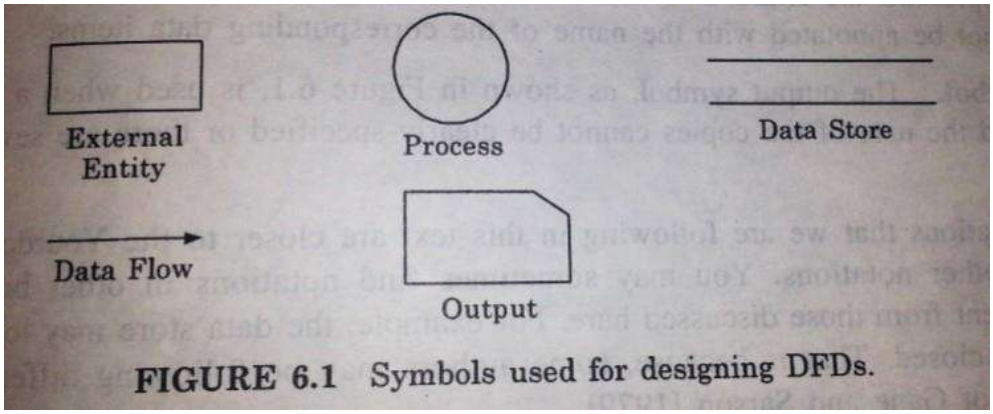


FIGURE 6.1 Symbols used for designing DFDs.

simple to understand any hierarchical model of a system, different details of the system can be slowly introduced through different hierarchies. The data flow diagramming technique also follows a very simple set of initiative concepts and rules. We now first explain the different symbols and then elaborate the various concepts associated with building a DFD model of a system.

PRIMITIVE SYMBOLS USED FOR CONSTRUCTING DFDs:

Functional symbol:

- A functional symbol is represented using a circle. This symbol is called a process or a bubble. Bubbles are annotated with the names of the corresponding functions.

External entity set:

- An external entity such as a librarian, a library member, etc. is represented by a rectangle. The external entities are essential those physical entities external to the software system which interact with the system by inputting data to the system or by consuming the data produced by the system.
- In addition to the human users, the external entity symbols can be used to represent external hardware and software such as another application software.

Data flow symbol:

- A directed arc or arrow is used as a data flow symbol. A data flow symbol represents the data flow occurring between two process, or between an external entity and a process in the direction of the data flow arrow. Data flow symbols are usually annotated with the corresponding data names.

Data store symbol:

- A data store represents a logical file.
- It is represented using two parallel lines.
- A logical file can represent either a data store symbol which can represent either a data structure or a physical file on disk.
- Each data store is connected to a process by means of a data flows symbol. The direction of data flow arrow shows whether data is being read from or written into a data store.
- An arrow flowing in or out of a data store implicitly represents the entire data of the data store and hence arrows connecting to a data store need not be annotated with the name of the corresponding data items.

Output symbol:

- The output symbol is used when a hard copy is produced and the user of the copies cannot be clearly specified or there are several users of the output.
  - ✓ For example, the data store may look like a box with one end closed.

Some Important Concepts Associated with Designing DFDs

Before we describe how to construct the DFD model of a system, let us discuss some important concept associated with DFDs.

Synchronous and asynchronous operations

- If two bubbles are directly connected by a data flow arrow, then they are synchronous. This means that they operate at the same speed.
  - ✓ For example, the validate number bubble can start processing only after the read-number bubble has supplied data to it and the read-number bubble has to wait until the validate-number bubble has consumed its data.
- However, if two bubbles are connected through a data store, then the speeds of operations of the bubbles are independent. The data produced by a producer bubble gets stored in the data store. The producer may store several pieces of data items in the data store before the consumer bubble consumes any of them.

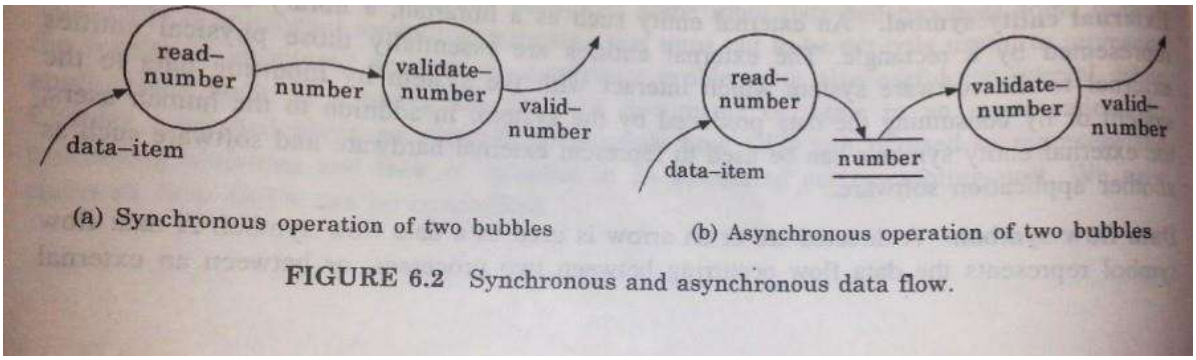


FIGURE 6.2 Synchronous and asynchronous data flow.

Data di cti onary

- Every DFD model of the system must be accompanied by a data dictionary.
- A data dictionary lists all data items appearing in the DFD model of the system.
- The data i tems l i sted i nclude all data flows and content of all data stores appearing on all DFDs in the DFD model of the system.
- I t may be recalled that the DFD model of the system typi cally consi sts of several DFDs, namely level 0 DFD, level 1DFD, level 2 DFDs, etc.
- However a single data dictionary should capture all the data appearing in all the DFDs consti tuting the DFD model of the system.
- A data dictionary lists the purpose of all data items and the definitions of all composite data i tems in terms of their component data items.

  ✓ For example, a data dictionary entry may represent that the data grosspay consists of the components regularPay and overtimePay.

Grosspay = regularPay + overti mePay

For smallest unit of the data item, the data dictionary lists their name and their type. The operator using a composite data item can be expressed in terms of their component data items.

A data dictionary plays a very important role in any software development process because of the following reason

- A data dictionary provides a standard terminology for all relevant data for use by all engineers working i n the same project. A consistent vocabulary for data items is very important, since in large projects di fferent engineers have a tendency to use different terms to refer to the same data, which unnecessarily causes confusion.
- The data dictionary provides the analyst with a means to determine the definition of different data structures in terms of their component elements.

For large system, the data dictionary can be extremely complex and voluminous.

Even moderate-sized projects can have thousands of entries in the data dictionary manually.

Computer-Aided Software Engineering (CASE) tools come handy to overcome this problem. Most CASE tools usually capture the data items appearing in a DFD, and automatically generate the data dictionary. These tools also supports some query language facility to raise the queries about the dictionary and usage of data items. For example, query may be formulated either to determine which data items affect which processes, which processes affect which data items, or to find the definition and usage of specific data items, and so on.

Query handling facilitated by storing data dictionary in a Relational Database Management System (RDBMS).

Data definition:

Composite data items can be defined in terms of primitive data items using the following data definition operators.

+: denotes composition of two data items, e.g. a+b represents data a and b

[, ,]: represents selection, i.e. any one of the data items listed inside the square bracket can occur. For example, [a,b] represents either a occurs or b occurs.

(): the content inside the bracket represent optional data which may or may not appear a+(b) represents that either a occurs or a+b occurs.

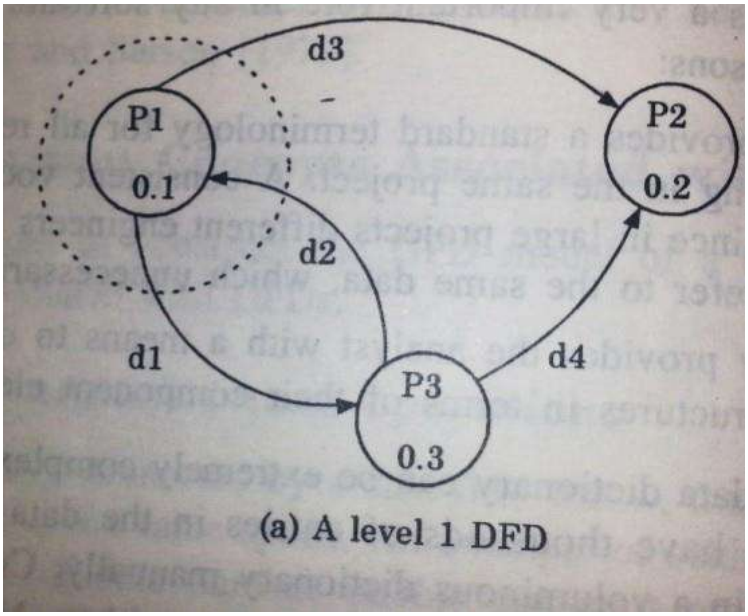
{}: represents iterative data definition, e.g. {name}5 represents five name data, {name} * represents zero or more instances of name data.

=: represents equilance, e.g. a=b+c means that a represents b and c

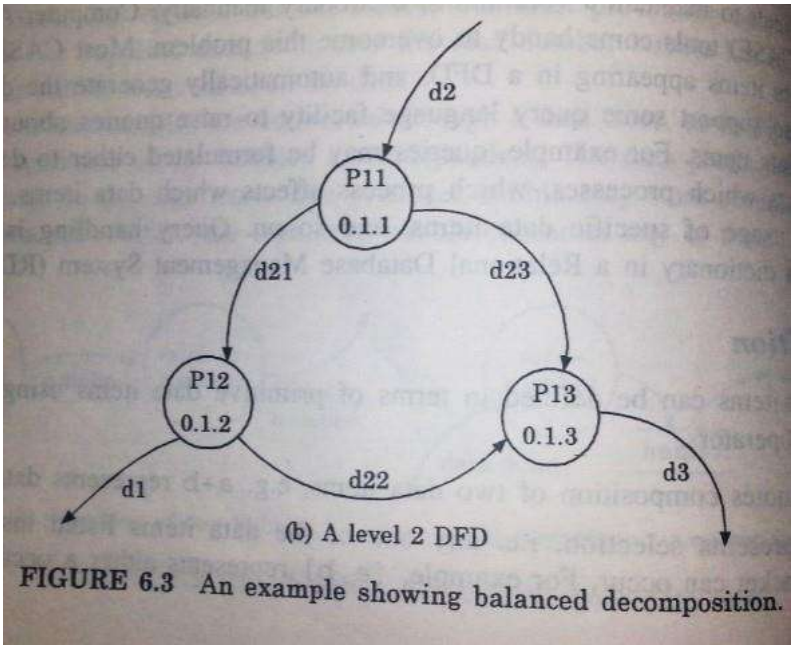
/* */: anything appearing within /* and */ is considered as commend

Balancing DFD's

- The data that flow into or out of a bubble must watch the data flow at the next level of DFD. This is known as balancing a DFD.
- In the level 1 of the DFD, data items d1 and d3 flow out of the bubble 0.1 and the data item d2 flows into the bubble P1(shown by the dotted circle). In the next level, bubble 0.1 is decomposed. The decomposition is balanced as d1 and d3 flow out of the level 2 diagram and d2 flown in.
- Note that the dangling arrows represent the data that flows into or out of the diagram.



(a) A level 1 DFD



(b) A level 2 DFD

FIGURE 6.3 An example showing balanced decomposition.

Developi ngthe DFD model of a system

- A DFD model of the system graphically depicts the transformation of the data input in the system to the final result through a hierarchy of level.
- A DFD starts with the most abstract definition of the system (lowest level) and at each higher level DFD, more details are successively introduced. To develop a higher level DFD model processes are decomposed into their sub process and the data flow among these sub processes are identified.
- To develop the data flow model of the system, first the most abstract representation of the problem is worked out. The most abstract representation

of the problem is also called context diagram. After developing, the context diagram the higher level DFDs are developed.

Context diagram

- The context diagram is the most abstract data flow representation of the system.
- It represents the entire system as a single bubble. This bubble is labelled according to the main function of the system.
- The various external entities with which the system interact and the data flows occurring between the system and external entities are also represented.
- The data input to the system and the data output from the system are representing as incoming and outgoing arrows.
- The data flow across should be annotated with the corresponding data name. the name context diagram is well justified because it represent the context in which the system is to exist, i.e. the external entities (user) who would interact with the system and the specific data items they would be supplying to the system and the data items they would be receiving from the system. The context diagram is also called the level 0 DFD.
- To develop the context diagram of the system, we have to analyze the SRS document to identify the different types of users who would be using the system and the kinds of data they would be inputing to the system and the data they would be receiving from the system. Here, the team users of the system also includes the external system which supply data to or receive data from the system.
- The bubble in the context diagram is annotated with the name of the software system being developed (usually a noun). This is in contrast with the bubbles in all other level which are annotated with the verbs. This is to be expected since the purpose of the context diagram is to capture the context of the system rather than its functionality.

Level 1 DFD

- To develop the level 1 DFD, examine the high-level functional requirements. If there are between three to seven high-level functional requirements, then these can be directly represented as bubbles in the level 1 DFD. We can then examine the input data to these function and the data output by these functions and represent them appropriately in the diagram.

- If the system has more than seven high- level requirements, then some of the related requirements have to be combined and represented in the form of the bubble in the level 1 DFD. These can be split in the lower DFD levels. If a system has less than three high-level functional requirements, then some of the high-level requirement need to be split into their sub functions so that we have roughly about five to seven bubbles on the diagram.

Decomposition:

- ✓ Each bubble in the DFD represents a function performed by the system. The bubbles are decomposed into sub functions at the successive levels of the DFD. Decomposition of a bubble is known as factoring or exploding a bubble.

- ✓ Each bubble at any level of DFD is usually decomposed to anything between three to seven bubbles. Too few bubbles at any level make that level superfluous.

- ✓ For example, if a bubbles is decomposed to just one bubble or two bubbles, then this decomposition becomes redundant. Also, too many bubbles, i.e. more than seven bubbles at any level of the DFD make the DFD model hard to understand. Decomposition of the bubble should be carried on until a level is reached at which the function of the bubble can be described using a simple algorithm.

- ✓ We can now describe how to go about developing the DFD model system more systematically.

3. The SRS document is examined to determine:

   - Different high-level function that the system needs to perform
   - Data input to every high-level function
   - Data output from every high-level function
   - Interactions (data flow) among the identified high-level functions

   These aspects of the high-level function are then represented in a diagrammatic form. The form the top-level data flow diagram (DFD), usually called the DFD 0.

4. The high-level function described in the SRS document are examined. If there are between three to seven high-level requirements in the SRS document, then each of the high-level function can be represented in the form of a bubble. If there are more than seven bubbles, then some of them have to be combined. If there are less than three bubbles, then some of these have to be split.

5. Each high-level function is decomposed into its constituent sub-function through the following set of activities:

- Different sub-function of the high-function are identified
- Data input to each of these sub-function is identified.
- Data output from each of these sub-function are identified
- Interactions(data flow) amongthese sub-function are identified These

aspectsarethen representedinadiagrammaticform DFD.

6. Step 3 i s repeated recursi vely for each sub-functi on unti l a sub-functi on can be represented by usi nga simple algorithm.

Numberi ngof bubbles

It is necessary to number the different bubbles occurring in the DFD to uniquely identify all bubble in the DFD. The bubble at the context level is usually assigned the number 0 to i ndi cate that i t i s the 0 level DFD. Bubbles at level 1 are numbered 0.1,0.2,0.3,etc.

When a bubble numbered x is decomposed, its children bubble are numbered x1,x2,x3,etc. in this numbering scheme, by looking at the number of the bubble we can unambiguously determine its level, its ancestors and its successors.

***9*. What are the characteristics of the good design? And describe different types of coupling and cohesi on. (Nov 2017)**

The goal of software engineering is to design and develop better software. Six of the most important quality characteristics are maintainability, correctness, reusability, reliability, portability, and efficiency.

Maintainability : The easewith which changescan bemadeto satisfy new requirements or to correct defici encies.

Correctness : The degree with which software adheres to i ts speci fied requirements.

R eusabi l i ty : The ease with which software can be reused i n developi ngother software.

Reliability : The frequencyandcriticalityofsoftwarefailure, wherefailureisan unacceptable effect or behavior occurringunder permissible operatingconditions.

Portability : The easewith whichsoftwarecan beusedoncomputer configurationsother than its current one.

Efficiency : The degreewith whichsoftwarefulfills itspurposewithout wasteofresources. For coupli

ngand cohesionrefer questionnumber : 2 page number : 15

***10*. a) Explai n about structures analysi s. (6) (Nov 2017)**

The structured analysis technique uses function-based decomposition while modeling the problem. It focuses on the functions performed in the problem domain and the data consumed and produced by these functions.

The structured analysis method helps the analyst decide what type of information to obtain at different points in analysis, and it helps organize information so that the analyst is not overwhelmed by the complexity of the problem.

It is a top-down refinement approach, which was originally called structured analysis and specification and was proposed for producing the specifications. However, we will limit our attention to the analysis aspect of the approach. Before we describe the approach, let us describe the data flow diagram and data dictionary on which the technique relies heavily.

Data Flow Diagrams and Data Dictionary

Data flow diagrams (also called data flow graphs) are commonly used during problem analysis. Data flow diagrams (DFDs) are quite general and are not limited to problem analysis for software engineering discipline. DFDs are very useful in understanding a system and can be effectively used during analysis.

A DFD shows the flow of data through a system. It views a system as a function that transforms the inputs into desired outputs. Any complex system will not perform this transformation in a "single step", and data will typically undergo a series of transformations before it becomes the output.

The DFD aims to capture the transformation that take place within a system to the input data so that eventually the output data is produced. The agent that performs the transformation of data from one state to another is called a process (or a bubble). So, a DFD shows the movement of data through the different transformations or processes in the system.

The processes are shown by named circles and data flows are represented by named arrows entering or leaving the bubbles. A rectangle represents a source or sink and is a pet originator or consumer of data. A source or sink is typically outside the main system of study. An example of a DFD for a system that pays workers.

In this DFD there is one basic input data flow, the weekly timesheet, which originates from the source worker. His basic output is the paycheck, the sink for which is also the worker, In this system, first the employee's record is retrieved, using the employee ID, which is contained in the timesheet.

From the employee record, the rate of payment and overtime are obtained. These rates and the regular and overtime hours (from the timesheet) are used to compute the pay. After the total pay the tax-rate file is used. The amount of tax deducted is recorded in the employee and company records. Finally, the payback is issued for the net pay. The amount paid is also recorded in company records.

This DFD is an abstract description of the system for handling payment. It does not matter if the system is automated or manual. This diagram could very well be for a manual system where the computations are all done with calculators and not represented in this DFD.

For example, what happens if the error in the weekly timesheet is not shown in this DFD. This is done to avoid getting bogged down with details while constructing a DFD for the overall system. If more details are desired, the DFD can be further refined.

It should be pointed out that a DFD is not a flowchart. A DFD represents the flow of data. While a flowchart shows the flow of control, a DFD does not represent procedural information. So, while drawing a DFD, one must not get involved in procedural details, and procedural thinking must be consciously avoided.

For example, considerations of loops and decisions must be ignored. In drawing the DFD, the designer has to specify the major transforms in the path of the data flowing.

While analyzing the problem domain the problem can be partitioned with respect to its functionality or with respect to objects. Object oriented modeling for (object-oriented analysis) uses the latter approach. During analysis, an object represents some entity or some concept in the problem domain.

An object contains some state information and provides some services to entities outside the objects. The state of the object can be accessed or modified only through the services they provide. Some objects also interact with the users through their services such that the users get the desired services.

Hence the goal of modeling is to identify the objects that exist in the problem domain, define the objects by specifying what state information they encapsulate and what services they provide, and identify relationships that exist between objects, such that the overall model is such that it supports the desired user services. Such a model of a system is called its object model.

10. b) write short notes on structured and detailed design. (5) (Nov 2017)

Structured Design: Structured design was developed by Constantine as a top-down technique for architectural design of software system. The basic approach in structured design is systematic conversion of data flow diagrams into structure charts. Design heuristics such as coupling and cohesion are used to guide the design process.

The first step in structured design is review and refinement of the data flow diagram developed during requirements definition and external design. The second step is to determine whether the system is transform centered or transaction-driven, and to derive a

high level structure chart based on this determination.

The third step in structured design is decomposition of each subsystem using guidelines such as coupling, cohesion, information hiding, and levels of abstraction, data abstraction, and the other decomposition criteria. The primary strength of structure design is provision of a systematic method for converting data flow diagrams into top-level structure charts.

Detailed design

- Interface design
  - module interfaces are negotiated, designed and documented
- Data structure and algorithm design
  - module details (data structures and algorithms) to provide system services are specified

When most people talk about detailed design, they are referring to a process known as top-down design. In short, when you think about the problem you are trying to solve, you start at the highest level and then work yourself into the details. This approach works very well when you have an overall structure you want your application to live within. At the macro level you are considering how many machines will be needed to host your application, which existing services you will need to use, etc. As you dive deeper, you are looking at use cases (or user stories if you prefer that terminology), and error handling (use cases have both normal and error condition paths to worry about). As you go even further into the details, you are looking at your algorithm, state transitions, logical sequence, and how internal parts of the code work together.